



Mosbat AI

COMMAND HQ — TECHNICAL DOCUMENTATION

AI Agent Control Plane · Codelgniter 4 + Supabase + Claude
Generated June 1, 2026

Contents

1. What Mosbat AI Is
2. How It Works (The Big Picture)
3. Technology Stack
4. The Message Flow — Step by Step
5. Features Overview
6. The Agent Tools
7. Code Walkthrough — Every File
8. Setup & Activation Checklist
9. Glossary

1. What Mosbat AI Is

Mosbat AI is a **command center for running companies with AI agents**. Instead of one chatbot, you have an organization of AI employees — a CEO, directors, and specialists — each with their own role, skills, and memory. You issue commands and they respond, delegate to each other, and take real actions.

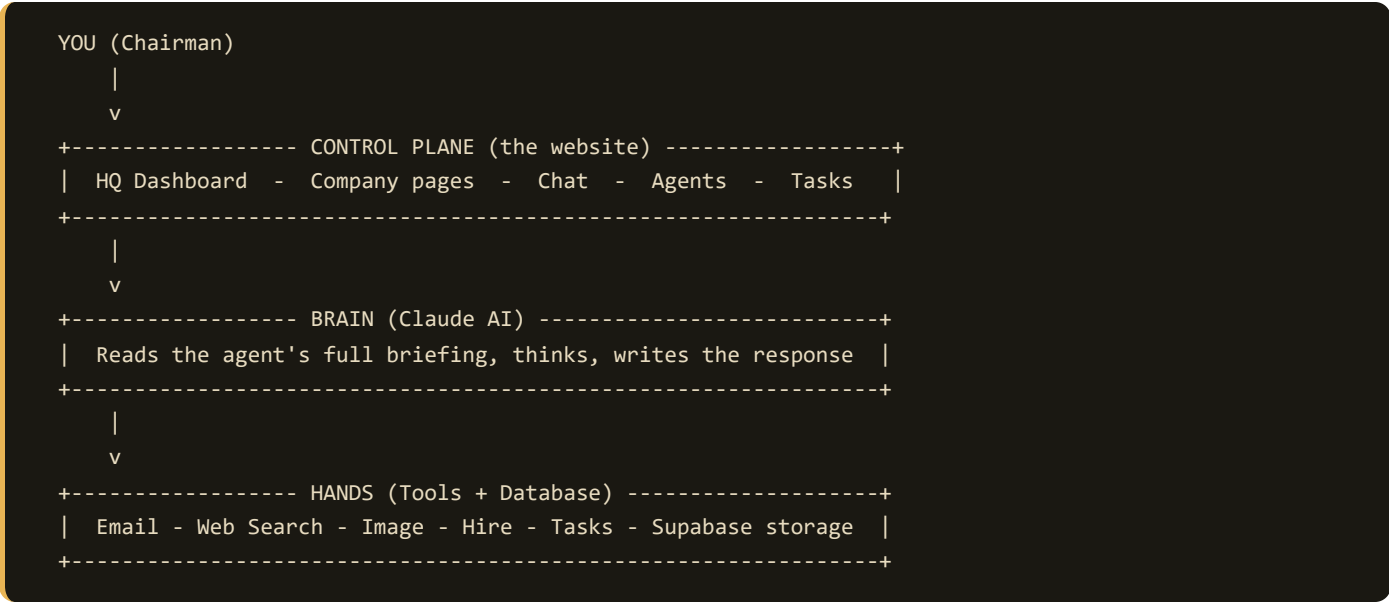
The simple analogy: Think of it like an office building. Each AI agent is an employee with a job description (their instructions), a manager (parent agent), and abilities (tools). You're the chairman — you give direction, and the org carries it out.

What makes it different from a normal chatbot

Normal chatbot	Mosbat AI
One assistant	A full org chart of specialized agents
Only talks	Sends real emails, searches the web, generates images, hires agents
Forgets everything	Remembers conversations (stored in database)
You do the work	Agents process tasks in the background autonomously

2. How It Works (The Big Picture)

There are three layers working together:



- **Control plane** — the web pages you click around in (built in PHP).
- **Brain** — Claude, the AI model that does the thinking.

- **Hands** — the tools and database that let agents take real actions and remember things.

3. Technology Stack

Layer	Technology	Role
Backend framework	CodeIgniter 4 (PHP)	Runs the website, routes, controllers
Database	Supabase (PostgreSQL + REST)	Stores agents, skills, conversations, tasks
AI model	Anthropic Claude	The intelligence behind every agent
Web search	Serper.dev	Real Google results for agents
Email	Hostinger SMTP	Agents send real emails
Images	Pollinations (free) / OpenAI DALL·E 3	Agents generate images
Frontend	HTML + Tailwind (CDN) + vanilla JS	The UI, no build step

4. The Message Flow — Step by Step

When you send a message in chat, here's exactly what happens:

1. You type a message and hit Send.
2. The browser calls `/api/chat/stream`
3. PHP builds the SYSTEM PROMPT (the agent's full briefing):
 - a) the agent's own instructions
 - b) the list of teammates it can delegate to
 - c) its assigned skills (extra knowledge)
 - d) the tool manual (email, search, image, hire...)
 - e) a "be brief" style rule
4. PHP loads the last 12 conversation turns (memory) from Supabase.
5. PHP sends the briefing + your message to Claude.
6. Claude streams its answer back word-by-word -> shown live in chat.
7. When Claude finishes, PHP scans the answer for tool tags.
8. Any tools found are EXECUTED (send email, generate image, etc.).
9. Tool results appear as cards in the chat.
10. The conversation is saved to Supabase for next time.

Key point: Building the briefing is pure PHP string work — Claude is only called *once* per message. Claude never builds its own prompt; it just receives the finished briefing and your question together.

5. Features Overview

HQ Dashboard /

- Welcome strip with time-based greeting and chairman identity.
- Metric tiles: companies, total agents, active agents.
- **Quick Command** — issue a directive to any company's CEO without leaving HQ.
- Portfolio of company cards (click to enter a company).
- Settings → global Skills manager (create/edit/delete skills).

Company Page /company/:id

- **Chat** — live streaming command channel with the company's director.
- **Dashboard** — metrics, recent tasks, active agents, full roster.
- **Inbox** — every assistant response captured (All / Unread).
- **Issues** — assigned tasks (from database) + session activity, with cancel/retry.
- **Agents** — grid of agent cards + a detail panel (Overview, Instructions, Skills, Runs).

Autonomous Tasks

Assign a task to an agent and a background job (`php spark agent:run`) picks it up, has the agent complete it using tools, and saves the output — all without you being present.

Skills System

Reusable knowledge blocks (context / instruction / tool / memory) you attach to agents. They get injected into the agent's briefing so it knows your business specifics.

Org Management (Paperclip-style)

- The CEO can **hire** new agents via a tool.
- The CEO can **fire** (deactivate) agents — except the top director.
- You can **delete** agents from the UI (children auto re-parent).

6. The Agent Tools

Agents take real actions by writing special tags in their response. PHP detects these tags after the response and executes them.

`[SEARCH]` **Web Search**

Gets real, current Google results. Powered by Serper.dev.

```
[SEARCH]
query: top community reward platforms 2025
[/SEARCH]
```

`[SEND_EMAIL]` **Send Email**

Sends a real email through Hostinger SMTP.

```
[SEND_EMAIL]
to: chairman@example.com
subject: Weekly Report
body:
Dear Chairman, here is this week's summary...
[/SEND_EMAIL]
```

`[GENERATE_IMAGE]` **Generate Image**

Creates a real image. Uses free Pollinations by default, or DALL·E 3 if an OpenAI key is set. The server downloads the image and hosts it locally so it always displays.

```
[GENERATE_IMAGE]
prompt: A mental health awareness illustration, warm gold and teal...
size: 1024x1024
[/GENERATE_IMAGE]
```

`[CREATE_TASK]` **Assign Task to an Agent**

Delegates work to another agent. The task is saved and the runner processes it.

```
[CREATE_TASK]
agent: Content Agent
title: Write 5 social posts about community growth
priority: high
description: Focus on positivity and our brand voice.
[/CREATE_TASK]
```

[HIRE_AGENT] Hire a New Agent

Creates a real new agent in the company org.

```
[HIRE_AGENT]
name: SEO Specialist
role: SEO Manager
reports_to: Marketing Director
prompt: You are the SEO Specialist responsible for...
[/HIRE_AGENT]
```

[FIRE_AGENT] Fire (Deactivate) an Agent

Deactivates an agent (reversible). Cannot fire the top director.

```
[FIRE_AGENT]
agent: Outreach Agent
[/FIRE_AGENT]
```


7. Code Walkthrough — Every File

Grouped by folder. Each entry explains what the file is for and how it works in plain words.

Controllers (handle web requests)

app/Controllers/Dashboard.php

Purpose: Serves the two main pages.

- `index()` — builds the HQ dashboard: chairman info, the list of companies, agent counts, then renders `dashboard.php`.
- `company($id)` — loads one company's agents (real ones from Supabase, or demo data), finds the root/director agent, then renders `company.php`.

app/Controllers/ChatApi.php

Purpose: The brain's gateway — all chat and agent operations.

- `stream()` — the main chat endpoint. Builds the system prompt, calls Claude with streaming, then runs any tools in the reply.
- `send()` — non-streaming chat (used by the agent detail "Direct Command").
- `composeSystemPrompt()` — assembles the agent briefing: role + team + skills + tool manual + brevity rules.
- `createAgent()` / `updateAgent()` / `deleteAgent()` — manage agents.
- `skills()`, `allSkills()`, `assignSkill()`, `removeSkill()`, skill CRUD — manage skills.
- `history()`, `agentHistory()` — load past conversations.

app/Controllers/TaskApi.php

Purpose: Manage autonomous tasks.

- `list($companyId)` — return all tasks for a company.
- `create()` — assign a new task to an agent (status starts as *pending*).
- `update()` — change a task's status/output.
- `cancel()` — cancel a task *and* write a signal file so a running job stops within 2 seconds.

Models (talk to the database)

app/Models/SupabaseModel.php

Purpose: The single doorway to Supabase. Wraps Supabase's REST API with simple PHP methods.

- Agents: `getOrg` , `getCompanyAgents` , `getAgentBySlug` , `getAgentById` , `createAgent` , `updateAgent` , `deleteAgent` .
- Skills: `getAgentSkills` , `getAllSkills` , `createSkill` , `updateSkill` , `deleteSkill` , `assignSkillToAgent` , `removeSkillFromAgent` .
- Conversations: `getRecentTurns` (memory window), `saveTurn` , `getAllAgentTurns` .
- Tasks: `createTask` , `getCompanyTasks` , `getPendingTasks` , `claimTask` (atomic lock), `updateTask` , `resetStuckTasks` , `getTaskById` .
- `getLastError()` — surfaces the real Supabase error when something fails.

Clever bit: `claimTask` updates a row only if it's still *pending*, so two background jobs can never grab the same task.

Libraries (the brain + the hands)

`app/Libraries/ClaudeService.php`

Purpose: Talks to Claude's API.

- `chat()` — send a prompt, get a full reply.
- `chatStream()` — stream the reply word-by-word. Also accepts an *abort callback* checked every 2 seconds, so a cancelled task stops mid-stream.
- `cancelSignalPath()` — the shared file path used for instant cancellation.

`app/Libraries/ToolDispatcher.php`

Purpose: The traffic controller for tools. Scans an agent's reply for tool tags and runs each one.

- `hasTools()` — quick check if any tool tags exist.
- `executeAll()` — find and run every tool, return the combined results.
- Routes to: SearchService, EmailService, ImageService, and Supabase (for tasks/hire/fire).
- `toolDocs()` — the tool manual injected into agent briefings.

`app/Libraries/SearchService.php`

Purpose: Calls Serper.dev and formats Google results (answer box, knowledge graph, top results, news) into clean text for the agent.

`app/Libraries/EmailService.php`

Purpose: Sends real emails via Hostinger SMTP using CodeIgniter's email library. Converts the agent's text into a clean HTML email.

app/Libraries/ImageService.php

Purpose: Generates images. Picks free Pollinations or paid DALL·E 3 automatically. Downloads the result *server-side* and saves it to `public/generated/` so the browser always loads a stable, same-origin file.

Commands (background jobs run from the terminal)**app/Commands/AgentRun.php**

Purpose: The autonomous worker. Run every minute by cron.

- Resets tasks stuck over 10 minutes (crash recovery).
- Grabs pending tasks, locks each one, builds the agent briefing.
- Runs a tool loop (up to 8 turns): Claude responds → tools run → results fed back → repeat until done.
- Checks for cancellation before each step. Saves the final output.

Run: `php spark agent:run` | Preview: `php spark agent:run --dry`

app/Commands/ToolsTest.php

Purpose: Quick test harness to verify web search and email work.

Run: `php spark tools:test`

Views (the pages you see)**app/Views/dashboard.php**

Purpose: The HQ dashboard UI — welcome strip, metric tiles, Quick Command, company portfolio, settings/skills modal. Dark theme with gold accents and a circuit-board background with rotating gears.

app/Views/company.php

Purpose: The full company workspace — sidebar nav and 5 sections (Chat, Dashboard, Inbox, Issues, Agents) plus the agent detail panel. Handles streaming chat, tool rendering (email/image/task cards), task management, skills, hire/fire/delete. Same gold-on-dark theme + circuit background.

Config & Database**app/Config/Routes.php**

Purpose: Maps URLs to controller methods (the two pages + all `/api/*` endpoints).

database/tasks_migration.sql

Purpose: Creates the `tasks` table (with status/priority rules, indexes, and an auto-updating timestamp). Run once in Supabase.

`.env`

Purpose: All secrets and settings — Claude key, Supabase keys, SMTP, Serper, OpenAI, memory window. Never commit this file publicly.

8. Setup & Activation Checklist

Step	What to do	Status
1	Set <code>anthropic.apiKey</code> in <code>.env</code>	required
2	Set <code>supabase.url</code> + <code>supabase.serviceKey</code>	required
3	Run <code>database/tasks_migration.sql</code> in Supabase SQL Editor	enables tasks
4	Add cron: <code>* * * * * php spark agent:run</code>	enables autonomy
5	Set <code>smtp.*</code> (Hostinger email)	email works
6	Set <code>serper.apiKey</code> (free at serper.dev)	enables web search
7	Image generation	works free (no key)
8	Make <code>public/generated/</code> writable on the server	for images on VPS

Tool status at a glance

Tool	Provider	Needs
Image generation	Pollinations (free) / DALL-E 3	nothing (free) / OpenAI key (upgrade)
Email	Hostinger SMTP	SMTP credentials (set)
Web search	Serper.dev	free API key
Tasks / Hire / Fire	Supabase	run the SQL migration

9. Glossary

Term	Plain meaning
System prompt	The full briefing handed to the AI before your question — role, team, skills, tools.
Agent	One AI employee with a name, role, instructions, and skills.
Skill	A reusable block of knowledge or behavior attached to agents.
Tool	A real action an agent can trigger (email, search, image, hire...).
Task	A unit of work assigned to an agent, processed in the background.
Runner	The background job (<code>agent:run</code>) that completes pending tasks.
Streaming	The AI's reply appearing word-by-word in real time.

Root / Director	The top agent of a company (no manager) — usually the CEO.
Memory window	The last N conversation turns sent to the AI so it remembers context.

MOSBAT AI · COMMAND HQ · TECHNICAL DOCUMENTATION